

Assignment 2

Query Tuning

Database Tuning

Group Name (A 18)
Krajinovic Ante ,12244629

April 26, 2026

Creating Tables and Indexes

SQL statements used to create the tables Employee, Student, and Techdept, and the indexes on the tables:

```
CREATE TABLE employee (  
  ssnnum int ,  
  name VARCHAR (30) ,  
  manager VARCHAR (30) ,  
  dept VARCHAR (30) ,  
  salary int ,  
  numfriends int ,  
  PRIMARY KEY ( ssnnum , name ));  
CREATE UNIQUE INDEX index_ssnnum_employee on employee ( ssnnum );  
CREATE UNIQUE INDEX index_name_employee on employee ( name );  
CREATE INDEX index_dept_employee on employee ( dept );  
CREATE TABLE student ( ssnnum int ,  
  name VARCHAR (30) ,  
  course VARCHAR (30) ,  
  grade int ,  
  PRIMARY KEY ( ssnnum , name ));  
CREATE UNIQUE INDEX index_ssnnum_student on student ( ssnnum );  
CREATE UNIQUE INDEX index_name_student on student ( name );  
CREATE TABLE techdept ( dept VARCHAR (30)  
  manager VARCHAR (30) ,  
  location VARCHAR (30) ,  
  PRIMARY KEY ( dept ));  
CREATE UNIQUE INDEX index_dept_techdept on techdept ( dept );
```

Populating the Tables

How did you fill the tables? What values did you use? Give a short description of your program.

Für das Befüllen der Tabellen wurden Listen mit vordefinierten Werten für Manager, Kurse und Noten erstellt. Zusätzlich existieren getrennte Listen für technische und nicht-technische Departments sowie eine HashMap, welche jedem Department einen Manager zuordnet.

Zunächst werden 20 nicht-technische Departments (z.B. Dept_A, Dept_B, ...) generiert. Jedem Department wird ein zufälliger Manager zugewiesen, und beide werden sowohl in einer Liste als auch in der HashMap gespeichert.

```
public static void mapDepts () {
    char c = "A";
    int deptCounter = 0;
    while ( deptCounter <= 20){
        String dept = " Dept_ " + c ;
        String manager = managers .get ( random . nextInt ( managers . size ()));
        mapDeptToManager . put ( dept , manager );
        depts .add( dept );
        deptCounter ++;
        c ++;
    }
}
```

Technische Departments werden analog erstellt, jedoch mit zusätzlich zufällig gewählten Locations. Die Daten werden effizient mittels Prepared Statements und Batch-Verarbeitung eingefügt, um die Anzahl der Datenbankzugriffe zu minimieren.

```
public static void fillTechdept () throws SQLException {
    String sqlStm = " INSERT INTO techdept (dept , manager , location )
VALUES ( ? , ? , ? )";
    PreparedStatement stm = con . prepareStatement ( sqlStm );
    int batchSize = 10;
    int batchCounter = 0;
    int insertCounter = 1;
    char c = "A";
    while ( insertCounter <= 10) {
        insertCounter ++;
        String dept = " Techdept_ " + c ;
        String manager = managers .get ( random . nextInt ( managers . size ()));
        String location = locations .get ( random . nextInt ( locations . size ()));
        mapDeptToManager . put ( dept , manager );
        techdepts .add( dept );
        stm . setString (1 , dept );
        stm . setString (2 , manager );
        stm . setString (3 , location );
        2
        stm . addBatch ();
        batchCounter ++;
        c ++;
        if ( batchCounter % batchSize == 0) {
            stm . executeBatch ();
        }
    }
    stm . executeBatch ();
}
```

Für die Employee-Tabelle wird die ssnnum beginnend bei 111111 in Zweierschritten erhöht, um eindeutige Werte zu garantieren. Der Name wird direkt aus der ssnnum abgeleitet. Anschließend wird mit einer Wahrscheinlichkeit von 10% ein technisches Department gewählt, ansonsten ein nicht-technisches. Der zugehörige Manager wird über die HashMap bestimmt. Zusätzlich werden zufällige Gehälter (zwischen 40.000 und 100.000) sowie eine zufällige Anzahl an Freunden (3–20) generiert.

```
public static void fillEmployee () throws SQLException {
```

```

String sqlStm = " INSERT INTO employee (ssnum , name , manager ,
dept , salary , numfriends ) VALUES (? ,? ,? ,? ,? ,?)";
PreparedStatement stm = con . prepareStatement ( sqlStm );
int batchSize = 100000;
int batchCounter = 0;
int insertCounter = 1;
int ssnum = 111111;
while ( insertCounter <= 100000) {
insertCounter ++;
String name = " Name_ " + ssnum ;
double techdeptChance = random . nextDouble ();
String dept ;
String manager ;
if ( techdeptChance <= 0.1){
dept = techdepts .get ( random . nextInt ( techdepts . size ()));
manager = mapDeptToManager .get ( dept );
} else {
dept = depts .get ( random . nextInt ( depts . size ()));
manager = mapDeptToManager .get ( dept );
}
int salary = random . nextInt (60000) + 40000;
int numFriends = random . nextInt (17) + 3;
stm . setInt (1 , ssnum );
stm . setString (2 , name );
stm . setString (3 , manager );
stm . setString (4 , dept );
stm . setInt (5 , salary );
3
stm . setInt (6 , numFriends );
stm . addBatch ();
batchCounter ++;
ssnum = ssnum + 2;
if ( batchCounter % batchSize == 0) {
stm . executeBatch ();
}
}
stm . executeBatch ();
}

```

Die Student-Tabelle wird analog zur Employee-Tabelle befüllt. Dabei werden zufällige Kurse und Noten (1–4) gewählt. Die ssnum startet bei 280000, um teilweise Überschneidungen mit Employee-Werten zu erzeugen, jedoch ohne vollständige Gleichheit.

```

public static void fillStudent () throws SQLException {
String sqlStm = " INSERT INTO student (ssnum , name , course , grade )
VALUES (? ,? ,? ,?)";
PreparedStatement stm = con . prepareStatement ( sqlStm );
int batchSize = 100000;
int batchCounter = 0;
int insertCounter = 1;
int ssnum = 280000;
while ( insertCounter <= 100000) {
insertCounter ++;
String name = " Name_ " + ssnum ;
String course = courses .get ( random . nextInt ( courses . size ()));
int grade = random . nextInt (4) + 1;
stm . setInt (1 , ssnum );
}
}

```

```

stm . setString (2 , name );
stm . setString (3 , course );
stm . setInt (4 , grade );
stm . addBatch ();
batchCounter ++;
ssnum = ssnum + 2;
if ( batchCounter % batchSize == 0) {
stm . executeBatch ();
}
}
stm . executeBatch ();
}

```

Queries

Query 1

Original Query Give the first type of query that might be hard for your database to optimize.

```

SELECT name
FROM employee e
WHERE salary > ( SELECT AVG( e2 . salary )
FROM employee e2 , techdept t
WHERE e2 . dept = e . dept
AND e2 . dept = t . dept )

```

Rewritten Query Give the rewritten query.

```

1.) Hilfstabelle erstellen
SELECT AVG ( e . salary ) as averageSalary , e . dept INTO Temp
FROM employee e , techdept t
WHERE e . dept = t . dept
GROUP BY e . dept
2.) Im Query verwenden
SELECT name
FROM employee e , temp te
WHERE salary > averagesalary AND e . dept = te . dept

```

1	Seq Scan on employee e (cost=0.00..105614576.00 rows=33333 width=12) (actual time=1.329..3755.881 rows=4980.00 loops=1)
2	Filter: ((salary)::numeric > (SubPlan 1))
3	Rows Removed by Filter: 95020
4	Buffers: shared hit=6314535
5	SubPlan 1
6	-> Aggregate (cost=1056.11..1056.12 rows=1 width=32) (actual time=0.037..0.037 rows=1.00 loops=100000)
7	Buffers: shared hit=6313609
8	-> Nested Loop (cost=41.44..1048.04 rows=3226 width=4) (actual time=0.008..0.034 rows=99.02 loops=100000)
9	Buffers: shared hit=6313609
10	-> Index Only Scan using index_dept_techdept on techdept t (cost=0.15..8.17 rows=1 width=78) (actual time=0.001..0.001 rows=0.10 loops=1..)
11	Index Cond: (dept = (e.dept)::text)
12	Heap Fetches: 9946
13	Index Searches: 100000
14	Buffers: shared hit=109946
15	-> Bitmap Heap Scan on employee e2 (cost=41.29..1007.62 rows=3226 width=11) (actual time=0.071..0.268 rows=995.54 loops=9946)
16	Recheck Cond: ((dept)::text = (e.dept)::text)
17	Heap Blocks: exact=6183771
18	Buffers: shared hit=6203663
19	-> Bitmap Index Scan on index_dept_employee (cost=0.00..40.49 rows=3226 width=0) (actual time=0.035..0.035 rows=995.54 loops=9946)
20	Index Cond: ((dept)::text = (e.dept)::text)
21	Index Searches: 9946
22	Buffers: shared hit=19892
23	Planning:
24	Buffers: shared hit=124 dirtied=2
25	Planning Time: 1.563 ms
26	Execution Time: 3756.713 ms

Evaluation of the Execution Plans

[EXPLAIN ANALYZE

SELECT name FROM employee e WHERE salary > (SELECT AVG(e2.salary)

FROM employee e2, techdept t WHERE e2.dept = e.dept AND e2.dept = t.dept)]

Give an interpretation of the execution plan, i.e., describe how the original query is evaluated.

Die Employee-Tabelle wird sequenziell gescannt. Für jedes Tupel wird überprüft, ob das Gehalt größer ist als das Ergebnis eines korrelierten Subqueries. Dieser Subquery wird für jedes Tupel erneut ausgeführt und berechnet den Durchschnitt des Gehalts innerhalb desselben Departments.

Der Subquery enthält eine Aggregatfunktion (AVG) und verwendet einen Nested Loop Join zwischen der Employee- und der Techdept-Tabelle. Dabei wird ein Index Only Scan auf Techdept sowie ein Bitmap Heap Scan (basierend auf einem Bitmap Index Scan) auf Employee verwendet. Obwohl Indizes genutzt werden, ist die wiederholte Ausführung des Subqueries der große Kostenfaktor.

Give the execution plan of the rewritten query.

	QUERY PLAN	
	text	
1	Hash Join (cost=3176.00..10883.29 rows=98333 width=12) (actual time=13.723..15.217 rows=4980.00 loops=1)	
2	Hash Cond: ((te.dept)::text = (e.dept)::text)	
3	Join Filter: ((e.salary)::numeric > te.averagesalary)	
4	Rows Removed by Join Filter: 4966	
5	Buffers: shared hit=927	
6	-> Seq Scan on temp te (cost=0.00..15.90 rows=590 width=110) (actual time=0.012..0.013 rows=10.00 loops=1)	
7	Buffers: shared hit=1	
8	-> Hash (cost=1926.00..1926.00 rows=100000 width=23) (actual time=13.527..13.528 rows=100000.00 loops=1)	
9	Buckets: 131072 Batches: 1 Memory Usage: 6434kB	
10	Buffers: shared hit=926	
11	-> Seq Scan on employee e (cost=0.00..1926.00 rows=100000 width=23) (actual time=0.011..5.633 rows=100000.00 loops=1)	
12	Buffers: shared hit=926	
13	Planning:	
14	Buffers: shared hit=239	
15	Planning Time: 1.615 ms	
16	Execution Time: 15.749 ms	

[EXPLAIN ANALYZE

```
SELECT name FROM employee e, temp te WHERE salary > averagesalary AND e.dept = te.dept]
```

Give an interpretation of the execution plan, i.e., describe how the rewritten query is evaluated.

Die temporäre Tabelle wird einmalig berechnet und enthält den Durchschnitt des Gehalts pro Department. Im Hauptquery werden anschließend die Employee-Tabelle und die temporäre Tabelle jeweils sequenziell gelesen.

PostgreSQL verwendet hier einen Hash Join, bei dem die kleinere temporäre Tabelle in eine Hash-Struktur überführt wird. Dadurch können passende Tupel effizient gefunden werden. Die Filterbedingung (salary bigger than averagesalary) wird anschließend angewendet.

Discuss, how the execution plan changed between the original and the rewritten query. In both the interpretation of the query plans and the discussion focus on the crucial parts, i.e., the parts of the query plans that cause major runtime differences.

Im Gegensatz zur ursprünglichen Anfrage wird die Aggregation nicht mehr für jedes Tupel neu berechnet. Dadurch entfällt der korrelierte Subquery sowie der Nested Loop. Stattdessen wird die Aggregation einmalig durchgeführt und anschließend effizient per Join verwendet. Dies reduziert die Anzahl der benötigten Operationen drastisch und verbessert die Performance erheblich.

Experiment Give the runtimes of the original and the rewritten query.

	Runtime [m-sec]
Original query	2827
Rewritten query	8
md Original query	169
mdb Rewritten query	46

Discuss, why the rewritten query is (or is not) faster than the original query.

Die rewritten Query ist deutlich schneller, da redundante Berechnungen vermieden werden. Im Original wird die AVG-Berechnung für jedes Tupel erneut ausgeführt, was zu einer sehr hohen Laufzeit führt. In der optimierten Version wird diese Berechnung nur einmal durchgeführt und das Ergebnis wiederverwendet.

Query 2

Original Query Give the second type of query that might be hard for your database to optimize.

```
[SELECT DISTINCT ssnnum
FROM Employee
WHERE dept = "Techdept_B"]
```

Rewritten Query Give the rewritten query.

```
[SELECT ssnnum
FROM Employee
WHERE dept = "Techdept_B"
GROUP BY ssnnum
]
```

Evaluation of the Execution Plans Give the execution plan of the original query.

```
[EXPLAIN ANALYZE
SELECT ssnnum
FROM Employee
WHERE dept = "Techdept_B"
]
```

```

+--+
| {
  "query_optimization": {
    "r_total_time_ms": 0.077092728
  },
  "query_block": {
    "select_id": 1,
    "cost": 0.166504335,
    "r_loops": 1,
    "r_total_time_ms": 0.151271429,
    "nested_loop": [
      {
        "table": {
          "table_name": "Employee",
          "access_type": "ref",
          "possible_keys": ["index_dept_employee"],
          "key": "index_dept_employee",
          "key_length": "123",
          "used_key_parts": ["dept"],
          "ref": ["const"],
          "loops": 1,
          "r_loops": 1,
          "rows": 1019,
          "r_rows": 1019,
          "cost": 0.166504335,
          "r_table_time_ms": 0.101828014,
          "r_other_time_ms": 0.045153032,
          "r_engine_stats": {
            "pages_accessed": 10
          },
          "filtered": 100,
          "r_total_filtered": 100,
          "attached_condition": "employee.dept = 'Techdept_B'",
          "r_filtered": 100,
          "using_index": true
        }
      ]
    ]
  }
}
```

```

MariaDB [tuning]> analyze select distinct ssnnum from employee where dept = 'Techdept_B';
+----+ select_type | table | type | possible_keys | key | key_len | ref | rows | r_rows | filtered | r_filtered | Extra
+----+-----+-----+-----+-----+-----+-----+----+-----+-----+-----+-----+-----+
1 | SIMPLE | employee | ref | index_dept_employee | index_dept_employee | 123 | const | 1019 | 1019.00 | 100.00 | 100.00 | Using where; Using index
+----+-----+-----+-----+-----+-----+-----+----+-----+-----+-----+-----+-----+
1 row in set (0.002 sec)

```

QUERY PLAN		text	
1	HashAggregate	(cost=988.40..998.03 rows=963 width=4) (actual time=0.769..0.819 rows=1019.00 loops=1)	
2	Group Key:	ssnum	
3	Batches:	1 Memory Usage: 89kB	
4	Buffers:	shared hit=637	
5	-> Bitmap Heap Scan on employee	(cost=11.76..985.99 rows=963 width=4) (actual time=0.138..0.675 rows=1019.00 loops=1)	
6	Recheck Cond:	((dept)::text = 'Techdept_B'::text)	
7	Heap Blocks:	exact=635	
8	Buffers:	shared hit=637	
9	-> Bitmap Index Scan on index_dept_employee	(cost=0.00..11.52 rows=963 width=0) (actual time=0.076..0.076 rows=1019.00 loops=1)	
10	Index Cond:	((dept)::text = 'Techdept_B'::text)	
11	Index Searches:	1	
12	Buffers:	shared hit=2	
13	Planning Time:	0.106 ms	
14	Execution Time:	0.870 ms	

Give an interpretation of the execution plan, i.e., describe how the original query is evaluated.

Zur Bestimmung eindeutiger ssnnum-Werte wird zunächst ein Bitmap Index Scan auf dem Index index_dept_employee durchgeführt, um alle Tupel mit dept = 'Techdept_B'

zu identifizieren. Anschließend werden diese mittels Bitmap Heap Scan aus der Tabelle geladen.

Danach wird eine HashAggregate-Operation ausgeführt, um Duplikate zu entfernen. Dabei werden die Ergebnisse nach ssnum gruppiert und nur eindeutige Werte ausgegeben. Diese Operation benötigt zusätzlichen Speicher, ist jedoch effizient für große Datenmengen.

Give the execution plan of the rewritten query.

```
[EXPLAIN ANALYZE
SELECT ssnum
FROM Employee
WHERE dept = "Techdept_B"
GROUP BY ssnum]
```

QUERY PLAN	
1	HashAggregate (cost=988.40..998.03 rows=963 width=4) (actual time=0.524..0.573 rows=1019.00 loops=1)
2	Group Key: ssnum
3	Batches: 1 Memory Usage: 89kB
4	Buffers: shared hit=637
5	-> Bitmap Heap Scan on employee (cost=11.76..985.99 rows=963 width=4) (actual time=0.109..0.434 rows=1019.00 loops=1)
6	Recheck Cond: ((dept)::text = 'Techdept_B'::text)
7	Heap Blocks: exact=635
8	Buffers: shared hit=637
9	-> Bitmap Index Scan on index_dept_employee (cost=0.00..11.52 rows=963 width=0) (actual time=0.062..0.062 rows=1019.00 loops=1)
10	Index Cond: ((dept)::text = 'Techdept_B'::text)
11	Index Searches: 1
12	Buffers: shared hit=2
13	Planning Time: 0.096 ms
14	Execution Time: 0.610 ms

Give an interpretation of the execution plan, i.e., describe how the rewritten query is evaluated.

Der Ausführungsplan entspricht weitgehend dem der davorigem Query. Auch hier wird zunächst ein Bitmap Index Scan und anschließend ein Bitmap Heap Scan durchgeführt, um relevante Tupel zu finden.

Die GROUP BY-Klausel führt ebenfalls zu einer HashAggregate-Operation, die funktional identisch zur DISTINCT-Operation ist. PostgreSQL verwendet intern denselben Mechanismus zur Duplikateliminierung.

Discuss, how the execution plan changed between the original and the rewritten query. In both the interpretation of the query plans and the discussion focus on the crucial parts, i.e., the parts of the query plans that cause major runtime differences.

Die Ausführungspläne beider Queries sind nahezu identisch. Sowohl DISTINCT als auch GROUP BY führen in diesem Fall zur gleichen internen Operation (HashAggregate). Daher unterscheiden sich die Laufzeiten nur minimal. Der einzige Unterschied wären die Planning und Execution Time.

Experiment Give the runtimes of the original and the rewritten query.

	Runtime [m-sec]
Original query	10
Rewritten query	1
md Original query	5
mdb Rewritten query	1

Discuss, why the rewritten query is (or is not) faster than the original query.

Die originale Query verwendet DISTINCT, um sicherzustellen, dass jeder ssnun nur einmal im Ergebnis vorkommt. Dafür muss die Datenbank doppelte Werte entfernen, was intern meist durch Sortieren oder Hashing passiert. Bei großen Tabellen ohne passenden Index kann das teuer sein, da viele oder alle Zeilen verarbeitet werden müssen.

Die umgeschriebene Query verwendet GROUP BY ssnun. In diesem Fall ist das Ergebnis gleich, da ebenfalls pro ssnun genau eine Gruppe gebildet wird. Auch GROUP BY erfordert in der Regel Sortier- oder Hash-Operationen.

Time Spent on this Assignment

Time in hours per person: **12**

References

Important: Reference your information sources!

Remove this section if you use footnotes to reference your information sources.
